

МИНОБРНАУКИ РОССИИ
САНКТ-ПЕТЕРБУРГСКИЙ ГОСУДАРСТВЕННЫЙ
ЭЛЕКТРОТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ
«ЛЭТИ» ИМ. В. И. УЛЬЯНОВА (ЛЕНИНА)
Кафедра МО ЭВМ

ОТЧЕТ
по лабораторной работе №7
по дисциплине «Вычислительная математика»
Тема: Решение систем линейных уравнений методом Гаусса.

Студент гр. 4342

Озернов Е.Н.

Преподаватель

Пуеров Г.Ю.

Санкт-Петербург

2026

Цель работы

Изучить и реализовать метод Гаусса для решения систем линейных алгебраических уравнений, включая выбор ведущего элемента, проверку совместности системы и контроль невырожденности матрицы в процессе вычислений; провести тестирование реализации на базовых, специальных (матрица Гильберта) и больших по размерности системах с анализом корректности, точности, времени и потребления памяти.

Теоретический минимум

Метод Гаусса применяется для решения системы линейных уравнений вида $A * x = b$, где A - матрица коэффициентов размером $m \times n$, x - вектор неизвестных размерности n , b - вектор правых частей размерности m . Идея метода состоит в том, чтобы с помощью элементарных преобразований строк привести систему к эквивалентному более простому виду, из которого неизвестные находятся последовательно.

На прямом ходе метода для каждого столбца выбирается ведущий элемент (pivot), после чего элементы ниже него зануляются по правилу: $R_i := R_i - (a_{ik} / a_{kk}) * R_k$. В результате матрица приводится к верхнетреугольному (или ступенчатому) виду. Чтобы уменьшить численные ошибки, обычно используют выбор ведущего элемента по максимальному модулю в текущем столбце (частичный выбор ведущего элемента).

После прямого хода выполняется обратный ход. Неизвестные вычисляются снизу вверх по формуле: $x_i = (b_i - \text{сумма по } j \text{ от } i+1 \text{ до } n (a_{ij} * x_j)) / a_{ii}$. Это дает решение при наличии нужного числа ненулевых ведущих элементов.

Проверка существования и числа решений выполняется через ранги. Если $\text{rank}(A) = \text{rank}([A|b])$, система совместна. Если при этом $\text{rank}(A) = \text{rank}([A|b]) = n$, решение единственное. Если $\text{rank}(A) = \text{rank}([A|b]) < n$, система имеет бесконечно много решений. Если $\text{rank}(A) \neq \text{rank}([A|b])$, система несовместна и решений нет. В процессе исключения это видно по специальным строкам: строка вида $[0 \ 0 \ \dots \ 0 \ | \ c]$, где $c \neq 0$, означает несовместность; строка $[0 \ 0 \ \dots \ 0 \ | \ 0]$ означает линейную зависимость и наличие свободных переменных.

Для квадратной матрицы важна проверка невырожденности. Эквивалентные условия невырожденности: $\det(A) \neq 0$, $\text{rank}(A) = n$, а также наличие n ведущих элементов при прямом ходе. Если $\det(A) = 0$ (или $\text{rank}(A) < n$), матрица вырождена.

Из-за вычислений с плавающей точкой в численных методах используется порог eps : элементы, модуль которых меньше eps , считаются нулевыми. Для оценки качества найденного решения используют невязку $r = A * x - b$ и ее норму $\|A * x - b\|$. Если известно точное решение x^* , дополнительно оценивают ошибку $\|x - x^*\|$.

Для плотной квадратной системы размером $n \times n$ вычислительная сложность метода Гаусса составляет примерно $O(n^3)$ по времени и $O(n^2)$ по памяти. Поэтому для больших размерностей важны эффективная реализация, выбор ведущего элемента и контроль численной устойчивости.

Выполнение работы

В ходе работы была реализована программа решения СЛАУ методом Гаусса на C++ с контролем корректности входа, выбором ведущего элемента, проверкой совместности/невырожденности в процессе вычислений, а также полноценным тестовым стендом с измерением времени и памяти.

1. Структура реализации

- gauss_solver.h

Содержит интерфейс решателя:

- PivotStrategy (сейчас реализована стратегия PartialColumnMax);
- SolutionType (Unique, Infinite, NoSolution, InvalidInput);
- GaussResult (тип решения, вектор x , ранг, признак невырожденности, текстовое сообщение);
- функция solveGaussian(...).
- gauss_solver.cpp
Содержит сам алгоритм метода Гаусса (прямой ход, проверка совместности, обратный ход, классификация результата).
- main.cpp
Консольный ввод системы (размеры, матрица, правая часть), вызов решателя, вывод результата.
- test.cpp
Набор тестов + вывод OK/FAIL по каждому тесту + генерация test_report.txt с входными данными, ответом, временем и памятью.

2. Формат ввода и вывода в основной программе

- Ввод в main.cpp:
 - m n
 - далее m строк матрицы A по n чисел
 - далее m чисел вектора b
- Программа выводит:

- текстовый статус (единственное, бесконечно много, нет решений, ошибка ввода);
- ранг;
- признак невырожденности;
- вектор x (если система совместна).

3. Алгоритм метода Гаусса, как реализовано

- Реализуется решение системы $A * x = b$.
- Функция `solveGaussian` принимает A и b по значению, чтобы работать с копией внутри функции и безопасно модифицировать строки.
- Пользователь может передавать `std::move(A)`, `std::move(b)` для снижения издержек на копирование больших структур.

Порядок работы алгоритма:

3.1. Проверка входа

- A не пустая;
- `b.size() ==` число строк A ;
- все строки A одинаковой длины (прямоугольная матрица);
- ограничение на размерность: $m \leq \text{max_dimension}$ и $n \leq \text{max_dimension}$ (по задаче — до 10000).

3.2. Прямой ход (элиминация)

- Идем по столбцам слева направо.
- Для текущего столбца `col` ищем ведущий элемент среди строк `row..m-1`.
- При частичном выборе ведущего элемента выбирается строка с максимальным `abs(A[i][col])`.
- Если `abs(A[sel][col]) <= eps`, ведущий элемент в столбце отсутствует, столбец пропускается.
- Иначе строки `sel` и `row` меняются местами.
- Для каждой строки ниже:
 - $\text{factor} = A[i][col] / A[\text{row}][col]$
 - $A[i][j] = A[i][j] - \text{factor} * A[\text{row}][j]$ для $j = \text{col}+1..n-1$

- $b[i] = b[i] - \text{factor} * b[\text{row}]$
- элемент $A[i][\text{col}]$ явно зануляется.
- На каждом найденном `pivot` увеличивается ранг.

3.3. Проверка совместности

- После прямого хода проверяются строки:
 - если строка $A[i]$ нулевая (все $\text{abs}(A[i][j]) \leq \text{eps}$), а $\text{abs}(b[i]) > \text{eps}$, то получено противоречие $0 = \text{неноль}$.
 - В этом случае сразу статус `NoSolution`.

3.4. Обратный ход

- Создается x длины n , по умолчанию нули.
- Для столбцов справа налево:
 - если в столбце нет `pivot` ($\text{where}[\text{col}] == -1$), переменная считается свободной и остается 0 (выбирается одно частное решение);
 - иначе вычисляется стандартной подстановкой:
 - $x[\text{col}] = (b[\text{row_of_pivot}] - \text{сумма известных членов}) / A[\text{row_of_pivot}][\text{col}]$.

3.5. Классификация результата

- `nonsingular = (m == n) и (rank == n)`.
- Если $\text{rank} < n$ и система совместна $\rightarrow$ `Infinite`.
- Иначе $\rightarrow$ `Unique`.

4. Выбор ведущего элемента

Реализован частичный выбор (максимум по модулю в текущем столбце).

Это ключевой момент устойчивости:

- снижает риск деления на очень малые числа;
- уменьшает рост округлительных ошибок;
- в большинстве практических задач дает хороший баланс между устойчивостью и скоростью.

5. Проверка невырожденности

Проверка невырожденности встроена в алгоритм:

- если матрица квадратная и найдено n ведущих элементов ($\text{rank} == n$), матрица невырождена;
 - иначе вырождена.
- Фактически это контроль условия $\text{rank}(A) = n$ прямо в ходе исключения.

6. Погрешность и численные аспекты

- Используется числовой порог $\text{eps} = 1e-12$.
- Все проверки “ноль/неноль” делаются через $\text{abs}(\text{value}) \leq \text{eps}$.
- Это критично для работы с вещественными числами, где точного нуля часто нет из-за округления.
- Тест на матрице Гильберта показывает, что реализация устойчива на численно трудном примере.
- По тестам:
 - для “простых” систем и диагональных больших матриц решение проходит с высоким запасом;
 - для Гильберта задан порог точности $\text{max_abs_err} < 1e-4$ (выполняется).
- Дополнительно (по отдельной проверке) невязка $\text{max}|A \cdot x - b|$ на Hilbert(8) получалась порядка $1e-16$, что является очень хорошим результатом для double.

7. Вычислительная сложность

Для плотной матрицы:

- Время:
 - квадратная $n \times n$: $O(n^3)$;
 - прямоугольная $m \times n$: $O(m * n * \min(m, n))$.
- Память:
 - матрица A : $O(m * n)$;
 - векторы b, x : $O(m + n)$;
 - служебные структуры (where): $O(n)$.

Для 10000 x 10000 плотной матрицы один только массив коэффициентов требует очень много памяти (порядка сотен МБ), что и наблюдается в метриках тестов.

Тестирование

Код тестов приведен в Приложении А. Тестирование приведено в таблице 1 в Приложении Б.

Вывод

В ходе работы был реализован и протестирован метод Гаусса для решения СЛАУ на C++ с частичным выбором ведущего элемента. Программа корректно определяет тип решения (единственное, бесконечно много, решений нет), выполняет проверку совместности системы и проверку невырожденности матрицы в процессе вычислений через ранг и наличие ведущих элементов. Проведенное тестирование на базовых примерах, матрице Гильберта и больших размерностях (1000x1000 и 10000x10000) подтвердило корректность реализации, достаточную численную устойчивость и работоспособность на больших входных данных. Дополнительно получены практические характеристики времени и памяти, что позволило оценить реальную вычислительную стоимость метода для крупных плотных систем.

ПРИЛОЖЕНИЕ А ИСХОДНЫЙ КОД ПРОГРАММЫ

Название файла: gauss_solver.h

```
#pragma once

#include <cstdint>
#include <string>
#include <vector>

// Стратегия выбора ведущего элемента.
// В этой лабораторной используется частичный выбор:
// в текущем столбце выбирается элемент с максимальным модулем.
enum class PivotStrategy {
    PartialColumnMax
};

// Тип результата решения СЛАУ.
enum class SolutionType {
    Unique,          // Единственное решение.
    Infinite,        // Бесконечно много решений.
    NoSolution,       // Решений нет (система несовместна).
    InvalidInput      // Некорректный вход: размеры, структура матрицы и т.д.
};

// Итог работы метода Гаусса.
struct GaussResult {
    SolutionType type = SolutionType::InvalidInput; // Классификация
результата.
    std::vector<double> x;                          // Найденный вектор
решения (если применимо).
    std::size_t rank = 0;                           // Ранг матрицы
коэффициентов.
    bool nonsingular = false;                        // Признак
невыврожденности (только для квадратной матрицы).
    std::string message;                            // Читаемое пояснение
результата.
};

// Решение СЛАУ  $A * x = b$  методом Гаусса.
// Важно:
// 1) A и b передаются по значению, чтобы функция могла изменять их "на
месте"
// без создания отдельной расширенной матрицы [A|b].
// Для больших задач желательно вызывать как
solveGaussian(std::move(A), std::move(b), ...),
// чтобы избежать лишнего копирования.
// 2) max_dimension задает рабочий предел по размерности задачи (по
условию 10000).
GaussResult solveGaussian(
    std::vector<std::vector<double>> A,
    std::vector<double> b,
    PivotStrategy pivot = PivotStrategy::PartialColumnMax,
    double eps = 1e-12,
    std::size_t max_dimension = 10000
);
```

Название файла: gauss_solver.cpp

```
#include "gauss_solver.h"

#include <algorithm>
#include <cmath>

namespace {

// Проверка, что строка матрицы имеет вид [0 0 ... 0],
// то есть все коэффициенты по модулю меньше eps.
bool isZeroRow(const std::vector<double>& row, double eps) {
    for (double value : row) {
        if (std::fabs(value) > eps) {
            return false;
        }
    }
    return true;
}

} // namespace

GaussResult solveGaussian(
    std::vector<std::vector<double>> A,
    std::vector<double> b,
    PivotStrategy pivot,
    double eps,
    std::size_t max_dimension
) {
    GaussResult result;

    // 1. Валидация входа.

    // Матрица не должна быть пустой.
    if (A.empty() || A[0].empty()) {
        result.type = SolutionType::InvalidInput;
        result.message = "Матрица коэффициентов не должна быть пустой.";
        return result;
    }

    const std::size_t m = A.size();
    const std::size_t n = A[0].size();

    // Размер вектора правой части должен совпадать с числом строк
    // матрицы коэффициентов.
    if (b.size() != m) {
        result.type = SolutionType::InvalidInput;
        result.message = "Размер вектора правой части должен совпадать с
числом строк матрицы коэффициентов.";
        return result;
    }

    // Матрица должна быть прямоугольной.
    for (const auto& row : A) {
        if (row.size() != n) {
            result.type = SolutionType::InvalidInput;
            result.message = "Матрица коэффициентов должна быть
прямоугольной (одинаковая длина всех строк).";
        }
    }
}
```

```

        return result;
    }
}

// По условию лабораторной поддерживаем системы до 10000x10000.
// Здесь ограничение накладывается отдельно на число строк и
столбцов.
if (m > max_dimension || n > max_dimension) {
    result.type = SolutionType::InvalidInput;
    result.message = "Размерность системы превышает допустимый
предел.";
    return result;
}

// where[col] = индекс строки, в которой стоит ведущий элемент для
col,
// либо -1, если в этом столбце ведущий элемент не найден.
std::vector<int> where(n, -1);

std::size_t row = 0; // Текущая рабочая строка (позиция для
следующего ведущего элемента).
std::size_t rank = 0; // Ранг считаем прямо в процессе прямого хода.

// 2. Прямой ход метода Гаусса (с выбором ведущего элемента).

for (std::size_t col = 0; col < n && row < m; ++col) {
    // ---- 2.1 Выбор ведущего элемента ----
    // По стратегии PartialColumnMax ищем в текущем столбце элемент
    // с максимальным модулем среди строк row..m-1.
    std::size_t sel = row;
    if (pivot == PivotStrategy::PartialColumnMax) {
        for (std::size_t i = row + 1; i < m; ++i) {
            if (std::fabs(A[i][col]) > std::fabs(A[sel][col])) {
                sel = i;
            }
        }
    }

    // Если в столбце нет значимого (по eps) элемента,
    // то ведущий элемент здесь отсутствует: столбец линейно зависим.
    // Это фиксирует вырожденность в процессе исключения.
    if (std::fabs(A[sel][col]) <= eps) {
        continue;
    }

    // ---- 2.2 Перестановка строк ----
    // Переносим выбранную строку на позицию текущей рабочей строки.
    std::swap(A[sel], A[row]);
    std::swap(b[sel], b[row]);
    where[col] = static_cast<int>(row);

    // ---- 2.3 Обнуление элементов ниже ведущего ----
    const double pivot_value = A[row][col];
    for (std::size_t i = row + 1; i < m; ++i) {
        const double factor = A[i][col] / pivot_value;

```

```

        // Если коэффициент практически нулевой, строка уже имеет
ноль в col.
        if (std::fabs(factor) <= eps) {
            A[i][col] = 0.0;
            continue;
        }

        // Явно обнуляем элемент под ведущим.
        A[i][col] = 0.0;

        // Обновляем оставшуюся часть строки.
        for (std::size_t j = col + 1; j < n; ++j) {
            A[i][j] -= factor * A[row][j];
        }

        // Обновляем правую часть.
        b[i] -= factor * b[row];
    }

    // После нахождения очередного ведущего элемента
    // увеличиваем ранг и переходим к следующей строке.
    ++row;
    ++rank;
}

// 3. Проверка совместности после прямого хода.

// Если встречается строка вида 0 ... 0 | c, где c != 0,
// система несовместна и решений нет.
for (std::size_t i = 0; i < m; ++i) {
    if (isZeroRow(A[i], eps) && std::fabs(b[i]) > eps) {
        result.type = SolutionType::NoSolution;
        result.rank = rank;
        result.nonsingular = false;
        result.message = "Система несовместна: обнаружена нулевая
строка с ненулевой правой частью.";
        return result;
    }
}

// 4. Обратный ход (восстановление решения).

std::vector<double> x(n, 0.0);

// Идем по столбцам справа налево.
for (int col = static_cast<int>(n) - 1; col >= 0; --col) {
    // Если в столбце нет ведущего элемента, это свободная
переменная.
    // Для получения одного конкретного решения оставляем x[col] = 0.
    if (where[static_cast<std::size_t>(col)] == -1) {
        continue;
    }

    const std::size_t r =
static_cast<std::size_t>(where[static_cast<std::size_t>(col)]);
    double sum = b[r];

```

```

        // Вычитаем уже найденные компоненты.
        for (std::size_t j = static_cast<std::size_t>(col) + 1; j < n;
++j) {
            sum -= A[r][j] * x[j];
        }

        x[static_cast<std::size_t>(col)] = sum /
A[r][static_cast<std::size_t>(col)];
    }

    // 5. Финальная классификация результата.

    // Невырожденность: только для квадратной матрицы и только при rank
== n.
    // Это условие проверяется по ходу работы через найденные ведущие
элементы.
    const bool nonsingular = (m == n && rank == n);
    result.rank = rank;
    result.nonsingular = nonsingular;
    result.x = std::move(x);

    // Если ранг меньше числа неизвестных, есть свободные переменные =>
бесконечно много решений.
    if (rank < n) {
        result.type = SolutionType::Infinite;
        result.message = "Система совместна и имеет бесконечно много
решений.";
        return result;
    }

    // Иначе решение единственное.
    result.type = SolutionType::Unique;
    result.message = "Система имеет единственное решение.";
    return result;
}

```

Название файла: main.cpp

```

#include "gauss_solver.h"

#include <iomanip>
#include <iostream>
#include <utility>
#include <vector>

int main() {
    // Формат ввода:
    // m n
    // a11 a12 ... a1n
    // ...
    // am1 am2 ... amn
    // b1 b2 ... bm
    std::size_t m = 0;
    std::size_t n = 0;

    std::cout << "Введите m и n: ";
    if (!(std::cin >> m >> n)) {

```

```

        std::cerr << "Ошибка ввода: не удалось прочитать размеры
матрицы.\n";
        return 1;
    }

    std::vector<std::vector<double>> A(m, std::vector<double>(n, 0.0));
    std::vector<double> b(m, 0.0);

    std::cout << "Введите матрицу A (" << m << " строк, " << n << "
столбцов):\n";
    for (std::size_t i = 0; i < m; ++i) {
        for (std::size_t j = 0; j < n; ++j) {
            if (!(std::cin >> A[i][j])) {
                std::cerr << "Ошибка ввода: не удалось прочитать A[" << i
<< "][" << j << "].\n";
                return 1;
            }
        }
    }

    std::cout << "Введите вектор b (" << m << " значений):\n";
    for (std::size_t i = 0; i < m; ++i) {
        if (!(std::cin >> b[i])) {
            std::cerr << "Ошибка ввода: не удалось прочитать b[" << i <<
"].\n";
            return 1;
        }
    }

    // Вызываем решатель. Для больших задач передаем данные через
std::move,
    // чтобы не делать лишних копий больших структур.
    const GaussResult result = solveGaussian(std::move(A), std::move(b));

    // Печатаем текстовое описание результата, ранг и признак
невырожденности.
    std::cout << result.message << "\n";
    std::cout << "Ранг: " << result.rank << "\n";
    std::cout << "Невырожденность: " << (result.nonsingular ? "да" :
"нет") << "\n";

    // Для совместных систем печатаем найденный вектор (для бесконечного
множества решений
    // это один из допустимых частных вариантов).
    if (result.type == SolutionType::Unique || result.type ==
SolutionType::Infinite) {
        std::cout << std::fixed << std::setprecision(8);
        for (std::size_t i = 0; i < result.x.size(); ++i) {
            std::cout << "x[" << i << "] = " << result.x[i] << "\n";
        }
    }

    return 0;
}

```

Название файла: test.cpp

```
#include "gauss_solver.h"
```

```

#include <algorithm>
#include <chrono>
#include <cmath>
#include <cstdint>
#include <fstream>
#include <iomanip>
#include <iostream>
#include <sstream>
#include <string>
#include <vector>

#ifdef _WIN32
#define PSAPI_VERSION 2
#include <windows.h>
#include <psapi.h>
#endif

namespace {

struct TestRunResult {
    bool ok = true;
    std::string input;
    std::string output;
    std::string error;
    double elapsed_ms = 0.0; // Время выполнения теста.
    std::uint64_t working_set_bytes = 0; // Рабочий набор после теста.
    std::uint64_t peak_delta_bytes = 0; // Прирост пикового рабочего
набора в тесте.
};

struct MemorySnapshot {
    std::uint64_t working_set_bytes = 0;
    std::uint64_t peak_working_set_bytes = 0;
};

// Сравнение чисел с учетом допуска eps.
bool approxEqual(double a, double b, double eps = 1e-7) {
    return std::fabs(a - b) <= eps;
}

// Проверка условия внутри теста: если условие ложно,
// тест помечается как неуспешный и сохраняется текст ошибки.
void expect(bool condition, const std::string& message, TestRunResult&
run) {
    if (!condition && run.ok) {
        run.ok = false;
        run.error = message;
    }
}

// Преобразование типа результата решателя в читаемый русский текст.
std::string solutionTypeToText(SolutionType type) {
    switch (type) {
        case SolutionType::Unique:
            return "Единственное решение";
        case SolutionType::Infinite:
            return "Бесконечно много решений";
        case SolutionType::NoSolution:

```

```

        return "Решений нет";
    case SolutionType::InvalidInput:
        return "Некорректный ввод";
    }
    return "Неизвестный статус";
}

// Чтение текущего и пикового потребления памяти процесса.
MemorySnapshot getMemorySnapshot() {
    MemorySnapshot snapshot;
#ifdef _WIN32
    PROCESS_MEMORY_COUNTERS counters{};
    if (K32GetProcessMemoryInfo(GetCurrentProcess(), &counters,
sizeof(counters)) != 0) {
        snapshot.working_set_bytes =
static_cast<std::uint64_t>(counters.WorkingSetSize);
        snapshot.peak_working_set_bytes =
static_cast<std::uint64_t>(counters.PeakWorkingSetSize);
    }
#endif
    return snapshot;
}

// Формат размера памяти в удобном виде.
std::string formatBytes(std::uint64_t bytes) {
    static const char* units[] = {"Б", "КБ", "МБ", "ГБ"};
    long double value = static_cast<long double>(bytes);
    int unit = 0;

    while (value >= 1024.0L && unit < 3) {
        value /= 1024.0L;
        ++unit;
    }

    std::ostringstream out;
    out << std::fixed << std::setprecision(2) <<
static_cast<double>(value) << " " << units[unit];
    return out.str();
}

// Краткая печать вектора: для больших размеров показываем только начало
и конец.
std::string formatVectorBrief(const std::vector<double>& v, std::size_t
edge = 3) {
    std::ostringstream out;
    out << std::fixed << std::setprecision(6);

    out << "[";
    if (v.size() <= 2 * edge) {
        for (std::size_t i = 0; i < v.size(); ++i) {
            if (i > 0) {
                out << ", ";
            }
            out << v[i];
        }
    } else {
        for (std::size_t i = 0; i < edge; ++i) {
            if (i > 0) {

```

```

        out << ", ";
    }
    out << v[i];
}

out << ", ..., ";

for (std::size_t i = v.size() - edge; i < v.size(); ++i) {
    if (i > v.size() - edge) {
        out << ", ";
    }
    out << v[i];
}
}
out << "]\n";

return out.str();
}

// Краткая печать одной строки матрицы: показываем только начало и конец
столбцов.
std::string formatRowBrief(const std::vector<double>& row, std::size_t
edge = 3) {
    std::ostringstream out;
    out << std::fixed << std::setprecision(6);

    out << "[";
    if (row.size() <= 2 * edge) {
        for (std::size_t j = 0; j < row.size(); ++j) {
            if (j > 0) {
                out << ", ";
            }
            out << row[j];
        }
    } else {
        for (std::size_t j = 0; j < edge; ++j) {
            if (j > 0) {
                out << ", ";
            }
            out << row[j];
        }

        out << ", ..., ";

        for (std::size_t j = row.size() - edge; j < row.size(); ++j) {
            if (j > row.size() - edge) {
                out << ", ";
            }
            out << row[j];
        }
    }
    out << "]\n";

    return out.str();
}

// Краткая печать матрицы: для больших размеров выводим только первые и
последние строки.

```

```

std::string formatMatrixBrief(const std::vector<std::vector<double>>& A,
std::size_t row_edge = 2, std::size_t col_edge = 3) {
    std::ostringstream out;
    const std::size_t m = A.size();
    const std::size_t n = A.empty() ? 0 : A[0].size();

    out << "Размер матрицы A: " << m << "x" << n << "\n";

    if (m == 0 || n == 0) {
        out << "[]\n";
        return out.str();
    }

    if (m <= 2 * row_edge) {
        for (std::size_t i = 0; i < m; ++i) {
            out << " строка " << i << ": " << formatRowBrief(A[i],
col_edge) << "\n";
        }
    } else {
        for (std::size_t i = 0; i < row_edge; ++i) {
            out << " строка " << i << ": " << formatRowBrief(A[i],
col_edge) << "\n";
        }

        out << " ... \n";

        for (std::size_t i = m - row_edge; i < m; ++i) {
            out << " строка " << i << ": " << formatRowBrief(A[i],
col_edge) << "\n";
        }
    }

    return out.str();
}

// Формирование текстового блока с входными данными теста.
std::string formatSystemBrief(const std::vector<std::vector<double>>& A,
const std::vector<double>& b) {
    std::ostringstream out;
    out << formatMatrixBrief(A);
    out << "Вектор b: " << formatVectorBrief(b) << "\n";
    return out.str();
}

// Формирование текстового блока с ответом решателя.
std::string formatSolverOutput(const GaussResult& result) {
    std::ostringstream out;

    out << "Статус: " << solutionTypeToText(result.type) << "\n";
    out << "Сообщение: " << result.message << "\n";
    out << "Ранг: " << result.rank << "\n";
    out << "Невырожденность: " << (result.nonsingular ? "да" : "нет") <<
"\n";

    if (result.type == SolutionType::Unique || result.type ==
SolutionType::Infinite) {
        out << "Вектор x: " << formatVectorBrief(result.x) << "\n";
    }
}

```

```

        return out.str();
    }

    // Генератор матрицы Гильберта размерности n.
    std::vector<std::vector<double>> makeHilbert(std::size_t n) {
        std::vector<std::vector<double>> h(n, std::vector<double>(n, 0.0));
        for (std::size_t i = 0; i < n; ++i) {
            for (std::size_t j = 0; j < n; ++j) {
                h[i][j] = 1.0 / static_cast<double>(i + j + 1);
            }
        }
        return h;
    }

    // Умножение матрицы на вектор: b = A * x.
    std::vector<double> matVec(const std::vector<std::vector<double>>& A,
    const std::vector<double>& x) {
        const std::size_t m = A.size();
        const std::size_t n = x.size();
        std::vector<double> b(m, 0.0);

        for (std::size_t i = 0; i < m; ++i) {
            for (std::size_t j = 0; j < n; ++j) {
                b[i] += A[i][j] * x[j];
            }
        }

        return b;
    }

    // Базовый тест: квадратная невырожденная система с единственным
    решением.
    TestRunResult testUniqueBasic() {
        TestRunResult run;

        std::vector<std::vector<double>> A{{2.0, 1.0}, {5.0, 7.0}};
        std::vector<double> b{11.0, 13.0};

        run.input = formatSystemBrief(A, b);

        const auto result = solveGaussian(A, b);
        run.output = formatSolverOutput(result);

        expect(result.type == SolutionType::Unique, "Ожидалось единственное
        решение", run);
        expect(result.nonsingular, "Матрица должна быть невырожденной", run);
        expect(approxEqual(result.x[0], 64.0 / 9.0, 1e-8), "Неверное значение
        x0", run);
        expect(approxEqual(result.x[1], -29.0 / 9.0, 1e-8), "Неверное
        значение x1", run);

        return run;
    }

    // Тест на несовместную систему (решений нет).
    TestRunResult testNoSolution() {
        TestRunResult run;

```

```

std::vector<std::vector<double>> A{{1.0, 1.0}, {1.0, 1.0}};
std::vector<double> b{1.0, 2.0};

run.input = formatSystemBrief(A, b);

const auto result = solveGaussian(A, b);
run.output = formatSolverOutput(result);

    expect(result.type == SolutionType::NoSolution, "Ожидался статус
«решений нет»", run);
    expect(!result.nonsingular, "Невырожденность невозможна при
несовместной системе", run);

    return run;
}

// Тест на бесконечно много решений.
TestRunResult testInfiniteSolutions() {
    TestRunResult run;

    std::vector<std::vector<double>> A{{1.0, 1.0}, {2.0, 2.0}};
    std::vector<double> b{2.0, 4.0};

    run.input = formatSystemBrief(A, b);

    const auto result = solveGaussian(A, b);
    run.output = formatSolverOutput(result);

    expect(result.type == SolutionType::Infinite, "Ожидался статус
«бесконечно много решений»", run);
    expect(!result.nonsingular, "Матрица должна быть вырожденной", run);

    return run;
}

// Тест на специальной матрице Гильберта (численно сложный пример).
TestRunResult testHilbert() {
    TestRunResult run;

    const std::size_t n = 8;
    auto H = makeHilbert(n);
    std::vector<double> x_true(n, 1.0);
    auto b = matVec(H, x_true);

    run.input = formatSystemBrief(H, b);

    const auto result = solveGaussian(H, b);
    run.output = formatSolverOutput(result);

    expect(result.type == SolutionType::Unique, "Ожидался статус
«единственное решение»", run);

    double max_abs_err = 0.0;
    for (std::size_t i = 0; i < n; ++i) {
        max_abs_err = std::max(max_abs_err, std::fabs(result.x[i] -
x_true[i]));
    }
}

```

```

    expect(max_abs_err < 1e-4, "Слишком большая ошибка восстановления
решения", run);

    return run;
}

// Большой тест 1000x1000.
TestRunResult testLarge1000() {
    TestRunResult run;

    const std::size_t n = 1000;
    std::vector<std::vector<double>> A(n, std::vector<double>(n, 0.0));
    std::vector<double> b(n, 0.0);

    for (std::size_t i = 0; i < n; ++i) {
        const double d = static_cast<double>(i + 1);
        A[i][i] = d;
        b[i] = d; // Истинное решение: x_i = 1.
    }

    run.input = formatSystemBrief(A, b);

    const auto result = solveGaussian(std::move(A), std::move(b),
PivotStrategy::PartialColumnMax, 1e-12, 10000);
    run.output = formatSolverOutput(result);

    expect(result.type == SolutionType::Unique, "Ожидался статус
«единственное решение»", run);
    expect(result.nonsingular, "Диагональная матрица с ненулевой
диагональю должна быть невырожденной", run);

    for (std::size_t i = 0; i < n; ++i) {
        expect(approxEqual(result.x[i], 1.0, 1e-8), "Неверная компонента
решения", run);
    }

    return run;
}

// Большой тест 10000x10000.
TestRunResult testLarge10000() {
    TestRunResult run;

    const std::size_t n = 10000;
    std::vector<std::vector<double>> A(n, std::vector<double>(n, 0.0));
    std::vector<double> b(n, 0.0);

    for (std::size_t i = 0; i < n; ++i) {
        A[i][i] = 1.0;
        b[i] = 1.0; // Истинное решение: x_i = 1.
    }

    run.input = formatSystemBrief(A, b);

    const auto result = solveGaussian(std::move(A), std::move(b),
PivotStrategy::PartialColumnMax, 1e-12, 10000);
    run.output = formatSolverOutput(result);

```

```

    expect(result.type == SolutionType::Unique, "Ожидался статус
«единственное решение», run);
    expect(result.nonsingular, "Единичная матрица должна быть
невыврожденной", run);

    for (std::size_t i = 0; i < n; ++i) {
        expect(approxEqual(result.x[i], 1.0, 1e-8), "Неверная компонента
решения", run);
    }

    return run;
}

// Запись информации по одному тесту в отчетный файл.
void writeTestReport(
    std::ofstream& report,
    std::size_t index,
    const std::string& name,
    const TestRunResult& run
) {
    report <<
    "=====\\n";
    report << "Тест №" << index << ": " << name << "\\n";
    report << "Входные данные:\\n";
    report << run.input;
    report << "Ответ решателя:\\n";
    report << run.output;
    report << "Время: " << std::fixed << std::setprecision(3) <<
run.elapsed_ms << " мс\\n";
    report << "Память (рабочий набор после теста): " <<
formatBytes(run.working_set_bytes) << "\\n";
    report << "Память (прирост пикового рабочего набора): " <<
formatBytes(run.peak_delta_bytes) << "\\n";
    report << "Итог теста: " << (run.ok ? "OK" : "FAIL") << "\\n";
    if (!run.ok) {
        report << "Причина: " << run.error << "\\n";
    }
    report << "\\n";
}

} // namespace

int main() {
    struct TestCase {
        const char* name;
        TestRunResult (*fn)();
    };

    const std::vector<TestCase> tests{
        {"Базовый уникальный случай", testUniqueBasic},
        {"Несовместная система", testNoSolution},
        {"Бесконечно много решений", testInfiniteSolutions},
        {"Матрица Гильберта", testHilbert},
        {"Большой тест 1000x1000", testLarge1000},
        {"Большой тест 10000x10000", testLarge10000}
    };
};

```

```

    std::ofstream report("test_report.txt", std::ios::out |
std::ios::trunc);
    if (!report.is_open()) {
        std::cerr << "Не удалось создать файл отчета test_report.txt\n";
        return 1;
    }

    std::size_t passed = 0;

    for (std::size_t i = 0; i < tests.size(); ++i) {
        const auto& test = tests[i];

        const MemorySnapshot mem_before = getMemorySnapshot();
        const auto time_begin = std::chrono::steady_clock::now();
        TestRunResult run = test.fn();
        const auto time_end = std::chrono::steady_clock::now();
        const MemorySnapshot mem_after = getMemorySnapshot();

        run.elapsed_ms = std::chrono::duration<double,
std::milli>(time_end - time_begin).count();
        run.working_set_bytes = mem_after.working_set_bytes;
        run.peak_delta_bytes = (mem_after.peak_working_set_bytes >
mem_before.peak_working_set_bytes)
            ? (mem_after.peak_working_set_bytes -
mem_before.peak_working_set_bytes)
            : 0;

        writeTestReport(report, i + 1, test.name, run);

        if (run.ok) {
            ++passed;
            std::cout << test.name << ": OK"
                << " | время: " << std::fixed <<
std::setprecision(3) << run.elapsed_ms << " мс"
                << " | память(пик+): " <<
formatBytes(run.peak_delta_bytes) << "\n";
        } else {
            std::cout << test.name << ": FAIL (" << run.error << ")"
                << " | время: " << std::fixed <<
std::setprecision(3) << run.elapsed_ms << " мс"
                << " | память(пик+): " <<
formatBytes(run.peak_delta_bytes) << "\n";
        }
    }

    std::cout << "Итог: " << passed << " из " << tests.size() << " тестов
пройдено.\n";
    std::cout << "Отчет сохранен в файл: test_report.txt\n";

    return (passed == tests.size()) ? 0 : 1;
}

```

ПРИЛОЖЕНИЕ В

ТЕСТИРОВАНИЕ

Таблица тестирования задачи:

№	Ввод (с описанием)	Вывод (включая решение)	Время	Память
1	Базовый уникальный случай: $A = \begin{bmatrix} 2 & 1 \\ 5 & 7 \end{bmatrix}$, $b = \begin{bmatrix} 11 \\ 13 \end{bmatrix}$ (квадратная невырожденная 2×2)	Единственное решение, $\text{rank} = 2$, невырожденность: да, $x = [7.111111, -3.222222]$	0.092 мс	140.00 КБ
2	Несовместная система: $A = \begin{bmatrix} 1 & 1 \\ 1 & 1 \end{bmatrix}$, $b = \begin{bmatrix} 1 \\ 2 \end{bmatrix}$	Решений нет, $\text{rank} = 1$, невырожденность: нет, x отсутствует	0.017 мс	4.00 КБ
3	Бесконечно много решений: $A = \begin{bmatrix} 1 & 1 \\ 2 & 2 \end{bmatrix}$, $b = \begin{bmatrix} 2 \\ 4 \end{bmatrix}$	Бесконечно много решений, $\text{rank} = 1$, невырожденность: нет, частное решение $x = [2.000000, 0.000000]$	0.016 мс	4.00 КБ
4	Матрица Гильберта: $n = 8$, $H(i,j) = 1/(i+j+1)$, $b = H * [1, \dots, 1]$	Единственное решение, $\text{rank} = 8$, невырожденность: да, $x \approx [1, 1, 1, 1, 1, 1, 1, 1]$	0.042 мс	0.00 Б

5	Большой тест 1000x1000: диагональная diag(1..1000), b = [1..1000]	Единственное решение, rank = 1000, невырожденность: да, x = [1,1,...,1] (1000 элементов)	7.000 мс	8.10 МБ
6	Большой тест 10000x10000: единичная I, b = [1,...,1]	Единственное решение, rank = 10000, невырожденность: да, x = [1,1,...,1] (10000 элементов)	1228.152 мс	756.08 МБ

Таблица 1 – Тестирование алгоритма.